Alexandra Tabita P.
24/543651/TK/60420
Assignment 3

Problem 1 : Snippet A

```
//n >= 1
int sum = 0;
for (int i=0; i<n; i++) {
    for (int j=0 =0; j<n; j++){
        sum += arr[i] + arr[j];
    }
}
```

(i) analyzing sequence

Line	Segment	Type	Cost	Time executed	Total Cost
2	int sum=0;	Assignment	1	1	1
3	int i=0;	Assignment	1	1	1
3	i<n;	Comparison	1	n+1	n+1
3	i++	Arithmetic+Assign	2	n	2n
4	int j=0;	Assignment	1	n	n
4	j<i;	Comparison	1	$\sum_{i=0}^{n-1}(i+1)$	$\frac{n(n+1)}{2}$
4	j++	Arithmetic+Assign	2	$\sum_{i=0}^{n-1}(i)$	$2 \times \left(\frac{n(n-1)}{2}\right)$
					$= n^2 - n$
5	sum+=arr[i]	2 Array Access,	5	$\sum_{i=0}^{n-1} i$	$5 \times \left(\frac{n(n-1)}{2}\right)$
	+arr[j];	2 Arith, 1 Assign			$= \frac{5n^2-5n}{2}$

Total cost of the operation T(n):

$T(n) = 1 + 1 + (n+1) + 2n + n + \frac{n^2+n}{2} + (n^2-n) + \frac{5n^2-5n}{2}$

$= 2 + 2 + 2n + 2 + 4n + 2n + n^2 + n + 2n^2 - 2n + 5n^2 - 5n$

$= \frac{8n^2 + n + 6}{2}$

$= 4n^2 + 0.5n + 3$

(ii) Big - O class :

↳ By dropping lower order terms and constant factors, Snippet A is $O(n^2)$

(iii) Best / worst case:

↳ The best and worst cases differ slightly in the number

There is no difference between best and worst cases because the loop always run for a fixed number of iterations regardless of the values in arr

Problem 1 : Snippet B

```
//n >= 1 ; arr is a sorted array of n integers
int lo = 0 , hi = n - 1 , target = 42;
bool found = false ;
while (lo <= hi ) {
    int mid = lo + (hi + lo) /2 ;
    if (arr[mid] == target ) { found = true ; break ; }
    else if (arr[mid] < target ) { lo = mid + 1 ; }
    else { lo = mid - 1 ; }
}
```

(i) Sequence analysis :

Line	Segment	Type	Cost	Time executed	Total cost
2	int lo = 0, hi = n	3 assigns, 1	4	1	4
	-1 , target = 42;	arithmetic			
3	bool found = false;	assignment	1	1	1
4	while (lo <= hi);	comparison	1	k + 1	k + 1
5	int mid = lo + (hi -	1 assignments	4	k	4k
	lo)/2 ;	3 arithmetic			
6	if (arr[mid] ==	1 access, 1	2	k	2k
	target)	computation			
8	else if (arr[mid]	1 access, 1	2	k	2k
	< target	computation			
10	lo = mid + 1	1 assigns, 1	2	k	2k
		arithmetic			
12	hi = mid - 1	1 assign ,	K	2K	
		1 arithmetic			

$T(n) \approx 11 \cdot \lceil \log_2 n \rceil + C$

(ii) Big - O class : $O(\log n)$

(iii) Best / worst case :

↳ They differ. The best case is $O(1)$ if the target is found at the first mid index. The worst case is $O(\log n)$ if the target is at the end of the search path

Problem 1: Snippet C

```
// n >= 1
int count = 0;
for (int i=0; i<n < i++ ){
    for (int j=0; j<n; j++ ) {
        for (int k=0; k<n; k+=2){  // note: k += 2
            if (arr[i] != arr[j] && arr[k] > 0)
                count ++;
        }
    }
}
```

(i) Sequence analysis:

Line	Segment	Type	cost	Times executed	Total cost
2	int count = 0;	Assignment	1	1	1
3	int i = 0;	assignment	1	1	1
3	i < n;	comparison	1	n+1	n+1
3	i ++	Arith + Assign	2	n	2n
4	int j = i+1;	Arith + Assign	2	n	2n
4	j < n;	comparison	1	$\left(\frac{n(n-1)}{2}\right)\left(\frac{n}{2}+1\right)$	$\frac{n^3}{4}$
4	j ++	Arith + Assign	2	$\left(\frac{n(n-1)}{2}\right)$	n^2-n
5	int k = 0;	Assignment	1	$\left(\frac{n(n-1)}{2}\right)$	$\frac{n^2-n}{2}$
5	k < n;	comparisons	1	$\left(\frac{n(n-1)}{2}\right)\left(\frac{n}{2}+1\right)$	$\frac{n^3}{4}$
5	k += 2	Arith + Assign	2	$\left(\frac{n(n-1)}{2}\right)\left(\frac{n}{2}\right)$	$\frac{n^3}{2}$
7	if (arr[i] != arr[j]	3 access,	6	$\frac{n(n-1)}{2} \times \left(\frac{n}{2}\right)$	$\frac{3n^3-3n^2}{2}$
	&& arr[k] > 0)	3 comp	2	Var(max: $\frac{n^3}{4}$)	$\frac{n^3}{2}$
8	count ++;	Arith + Assign			

Total cost of the operation T(n):

↳ dominated by the triple nested loop. Because the k loop jumps by 2 (k += 2) it runs $\frac{n}{2}$ times for every iteration

$T(n) \approx 2.5n^3 + An^2 + Bn + C$

(ii) Big - O class:

↳ applying asymptotic simplification (dropping lower-order terms and constant coefficient): $O(n^3)$

(iii) Best / worst case:

↳ They slightly differ. The execution of count ++ depends on the value in array. However, the asymptotic complexity $O(n^3)$ remains the same for both cases because the three loops will alway execute their full range regardless of the input data

KIKY

Dipindai dengan CamScanner

Alexandra Tabita P.
24/543651/TK/60420
Assignment 3

Problem 3 : Part B — Complexity Comparison Table :
Complete the following table by hand

operation	Best case	Worst case	Average case
Linear search (unsorted)	O(1)	O(n)	O(n)
Linear search (sorted, early exit)	O(1)	O(n)	O(n)
Binary (sorted)	O(1)	O(logn)	O(logn)

• Linear search (unsorted):
- Best case : O(1) cause the target might be the very first element checked at index 0
- Worst case : O(n) cause the target might not exist in the array, forcing the algorithm to check every single element
- Average case : O(n) cause on average, the target will be found halfway through the array, and checking n/2 elements drops the constant to become O(n)

• Linear search (sorted, early exit):
- Best case : O(1) cause the target could still be the very first element at index 0
- Worst case : O(n) cause if the target is larger than the maximum element in the array, the algorithm still must scan all n elements before confirming it is missing
- Average case : O(n) cause you still evaluate a fraction of n elements on average before finding the target or exceeding its possible value

• Binary search (sorted) :
- Best case : O(1) cause the target could be located at the exact midpoint on the very first calculation
- Worst case : O(logn) cause the algorithm repeatedly halves the search space until only 1 element remains, taking algorithmic steps
- Average case : O(logn) cause the most elements are found in the deepest levels of the search tree, requiring nearly the maximum number of halving steps

SiDU
Dipindai dengan CamScanner

Problem 4: Consider the pushBack function from Problem 2, which doubles the capacity whenever size == capacity, starting from capacity = 2

Problem 4: Part A

↳ In this strategy, the capacity starts at 2 and doubles (2, 4, 8, 16...) everytime the array reaches its limit

call #	value pushed	size after	capacity after	cost
1	10	1	2	1
2	20	2	2	1
3	30	3	4	3
4	40	4	4	1
5	50	5	8	5
6	60	6	8	1
7	70	7	8	1
8	80	8	8	1
9	90	9	16	9
10	10	10	16	1
11	110	11	16	1
12	120	12	16	1

Problem 4: Part B

(1) $\Sigma T_{12} = 1 + 1 + 3 + 1 + 5 + 1 + 1 + 1 + 9 + 1 + 1 + 1$

$= 26$

(2) Average cost $\Rightarrow \frac{T_{12}}{12} = \frac{26}{12} \approx 2,167$

(3) Total copy work across and pushbacks (Geometric Series)

Formula and evaluation: the copy occurs at 2, 4, 6, 8, ...

Total : $2 + 4 + 6 + 8 + \ldots + 2^k = \sum_{i=1}^{k} 2^i$

(2^k is the largest power of 2 strictly less than n)

this sum of geometric series is exactly $2^{k+1} - 2$

because $2^k < n \rightarrow 2^{k+1} \leq 2n$

Total copy is strictly less than 2n

SiDU

Dipindai dengan CamScanner

Problem 4 : Part C

(1)

call #2	value pushed	size after	capacity after	cost
1	10	1	2	1
2	20	2	2	1
3	30	3	6	3
4	40	4	6	1
5	50	5	6	1
6	60	6	6	1
7	70	8	10	7
8	80	7	10	1
9	90	9	10	1
10	100	10	10	1
11	110	11	14	11
12	120	12	14	1

(2) Compare Total Cost

from part B = 26

-> total increment C => 1 + 1 + 3 + 1 + 1 + 7 + 1 + 1 + 1 + 11 + 1 = 30

(3) -> Doubling strategy

(a) $\sum$ cost . $T(n) = n + O(n) = O(n)$

(b) Amortised cost :- $\frac{O(n)}{n} = O(1)$

-> Fixed Increment

(a) Resize : Terjadi di size c, c+k , c + 2k, ... (arithmatic)

(b) n approximation = n/k

(c) Total of copy work $B = \sum_{i=1}^{n/k} (i-k) = k \cdot \frac{n/k\,(n/k+1)}{2}$

$\approx \frac{n^2}{2k} = O(n^2)$

(d) Total of cost

$T(n) = n + O(n^2) = O(n^2)$

(e) Amortised cost $= \frac{O(n^2)}{n}$

$= O(n)$

SiDU

Dipindai dengan CamScanner